

```
foreach (int a in monTableau)
{
    Console.WriteLine(a);
}
```

3.2 Les Listes

Une liste peut être une alternative intéressante au tableau lorsque l'on ne connaît pas le nombre d'éléments à stocker à l'avance.

3.2.1 Déclaration d'une liste

Pour créer une liste rien de plus simple : on crée un objet de type liste grâce au mot clé « List » suivi du type d'éléments que l'on souhaite stocker entre crochets :

```
//C#
//Création d'une liste d'entiers :
List<int> MaListe1 = new List<int>();
//Création d'une liste de chaînes :
List<string> MaListe2 = new List<string>();
//...
```

3.2.2 Ajouter des éléments à la liste

A présent notre liste est créée, on va lui ajouter des valeurs, un vrai jeu d'enfant puisque les objets de type liste possède une méthode « Add() » qui nous permet d'ajouter des éléments entre parenthèses.

Dans l'exemple suivant, on va rajouter à notre liste les dix premiers nombres de la suite de Fibonacci :

```
//C#
static void Main(string[] args)
{
    List<int> Fibonacci = new List<int>();

    Fibonacci.Add(0);
    Fibonacci.Add(1);
    Fibonacci.Add(1);
    Fibonacci.Add(2);
    Fibonacci.Add(3);
    Fibonacci.Add(5);
    Fibonacci.Add(8);
    Fibonacci.Add(13);
    Fibonacci.Add(21);
    Fibonacci.Add(34);
}
```



Parcourir la liste

À présent nous allons parcourir notre liste et faire afficher chaque élément. Nous allons voir deux méthodes, une « traditionnelle » utilisant la boucle « `for` » et une seconde utilisant une boucle « `foreach` ».

3.2.3.1 Avec une boucle « `for` »

```
//C#

//Boucle "for" traditionnelle :
int i;
for (i = 0; i < Fibonacci.Count(); i++)
{
    Console.WriteLine(Fibonacci[i]);
}
```

3.2.3.2 Avec une boucle « `foreach` »

```
//C#

foreach (int element in Fibonacci)
{
    Console.WriteLine(element);
}
```

Cette boucle signifie « Pour chaque élément dans la liste Fibonacci afficher notre élément ». Le mot « `element` » a été choisi ici par défaut, mais vous pouvez le remplacer par n'importe quel mot de votre choix, à condition d'utiliser le même dans la partie de code de la boucle.

3.2.4 Action sur la liste

Les objets de type « `List` » possèdent de nombreuses méthodes qui nous permettent de modifier notre liste ou bien d'obtenir des informations sur les données stockées dans celles-ci. Nous allons présenter ici quelques exemples de méthodes utiles :

```
//C#

//Méthodes modifiant notre liste :

Fibonacci.Clear(); //Supprime toute les valeurs
Fibonacci.Insert(place,nb); //Insère notre nombre à la place désirée
Fibonacci.Reverse(); //Range la liste dans le sens inverse.

//Méthode descriptives :

Fibonacci.Count(); //Count retourne le nombre d'elements
Fibonacci.Max(); //Renvoie la valeur maximum de la liste
Fibonacci.Min(); //Renvoie la valeur minimum de la liste
Fibonacci.Sum(); //Renvoie la somme des éléments
Fibonacci.BinarySearch(nombre); //Renvoie l'indice de l'objet à trouver
Fibonacci.Contains(nombre); //Renvoie vrai si le nombre est trouvé
```




Les énumérations

Une énumération va nous permettre de créer notre propre type de variable afin d'en restreindre les valeurs possibles. Dans l'exemple suivant, nous allons prendre l'exemple des jours de la semaine.

3.3.1 Déclaration et initialisation

Premièrement, nous allons créer l'énumération « Jours » grâce au mot clé «enum» (Par convention, le nom d'une énumération est souvent écrit en PascalCase).

```
//C#  
  
enum Jours { lun, mar, mer, jeu, ven, sam, dim };
```

3.3.2 Utiliser une énumération

Ici nous allons poursuivre l'exemple en utilisant l'énumération précédemment déclarée. Dans notre main nous allons créer la variable « jourDeStage » qui sera du type « Jours » :

```
//C#  
  
class Program  
{  
    enum Jours { lun, mar, mer, jeu, ven, sam, dim };  
  
    static void Main(string[] args)  
    {  
        Jours jourDeStage; //variable de type "Jours"  
        jourDeStage = Jours.jeu;  
  
        Console.WriteLine(jourDeStage);  
        Console.ReadLine();  
    }  
}
```

Vous avez sûrement remarqué le fait que la déclaration de « Jours » se faisait hors de la méthode main. Ce détail est très important puisque la déclaration d'une énumération ne peut s'effectuer à l'intérieur d'une fonction.

Grâce à notre énumération, nous voyons ci-dessous que l'IntelliSense de Visual Studio nous propose les différents choix possibles pour notre variable « jourDeStage ».